

2D Motion Profiling Using Bézier Curves

Contents

1	What is Motion Profiling?	2
2	Our Goal: Move Along a Curve	2
3	Curve Representation Options	2
4	What are Keyframes?	2
5	Arc Length via Gaussian Quadrature	3
6	Finding the Next Parameter: Newton–Raphson	4
7	Acceleration and Deceleration Distances	5
8	Curvature and Turning Speed	5
9	Keyframes: Localisation and Velocity Interpolation	7
9.1	Locating a Keyframe on the Path	7
9.2	Interpolating Between Keyframes	8
10	Velocity Planning Algorithm	8
11	Handling Multiple Path Segments	9
12	Simplified C++ Example from VMPLib	10
13	Trajectory Tracking with RAMSETE	10
14	Why Acceleration/Deceleration Distance is an Approximation	11
15	Ideas for Future Improvement	12
16	Related Work and Similar Systems	12
17	Further Learning Resources	13
	Bibliography	14

1 What is Motion Profiling?

As the name suggest motion profiling profiles a motion, specifically deriving characteristics about a movement. Specifically it derives in most cases velocities and/or derivatives of it.

Motion profiling answers the question: how should a robot move from point A to point B?

- How fast should it go?
- When should it turn?
- When should it start slowing down?

In this case the result is a path plus a time-parameterized velocity and angular velocity profile that respects the robot's physical limits.

2 Our Goal: Move Along a Curve

We want to follow a curved path while:

- Respecting maximum speed and acceleration
- Slowing down for turns (based on curvature)
- Hitting specified speeds at key positions (keyframes)

To do that, we describe the path mathematically and then compute how the robot should move along it in real time.

3 Curve Representation Options

There are multiple ways to describe a smooth curve:

- **Quadratic Bézier Curve:** Uses 3 control points (a start point, end point, and one middle control point) to form a smooth curve. It is simple but less flexible in shaping the path.
- **Cubic Bézier Curve:** Uses 4 control points (start, end, and two control points). This gives more control over the curve's shape. We use cubic Béziars here for a good balance of simplicity and flexibility.
- **Cubic Hermite Splines:** Defined by endpoints and specified tangents (slopes) at those endpoints. This means you explicitly set the direction of travel at each end, resulting in a smooth curve that honors those directions.
- **Catmull-Rom Splines:** A type of spline that passes through a series of given waypoints. The curve is calculated to smoothly interpolate through all the points, making it easy to create paths through specific locations.

Here I chose the cubic Bézier for its balance of simplicity and control [1]. The following are the functions that describe the x and y coordinates of the cubic Bézier curve.

$$x(t) = (1-t)^3 x_0 + 3(1-t)^2 t x_1 + 3(1-t) t^2 x_2 + t^3 x_3$$

$$y(t) = (1-t)^3 y_0 + 3(1-t)^2 t y_1 + 3(1-t) t^2 y_2 + t^3 y_3$$

4 What are Keyframes?

Keyframes are specific points along the path where we fix the robot's speed. For example:

- Start at 0 m/s
- Reach 1 m/s halfway
- End at 0 m/s

The planner converts those into arc-length positions and then blends between them. In practice, path planning software (such as *PATH.JERRYIO* [2]) provides a visual interface to define keyframes. For example, *PATH.JERRYIO* includes a “speed graph” where you can add **speed keyframes** at various positions along the path. Each keyframe corresponds to a specific distance along the path (the x-axis on the graph) and a desired speed (the y-axis). The planner then constructs the speed profile by connecting these points smoothly, ensuring the robot slows down or speeds up at the designated locations.

5 Arc Length via Gaussian Quadrature

For a parametric curve $\mathbf{r}(t) = (x(t), y(t))$, the arc length from the start up to parameter t is

$$s(t) = \int_0^t \|\mathbf{r}'(\tau)\| d\tau,$$

the integral of speed $\|\mathbf{r}'(\tau)\|$ along the curve. We approximate this integral with Gaussian quadrature, following the approach used for arc-length parameterization of spline curves in real-time simulation [3]:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} \sum_{i=1}^n w_i f\left(\frac{b-a}{2}x_i + \frac{a+b}{2}\right).$$

Numerical integration rationale. For most curves, there isn’t a simple formula for $s(t)$ (and this is true for Cubic Bézier Curves), so we need to integrate numerically. Gaussian quadrature is one effective method for numerical integration. It chooses special sample points (nodes x_i) and weights w_i to maximize accuracy. An n -point Gauss–Legendre rule is exact for all polynomials up to degree $2n - 1$, meaning it integrates any such polynomial perfectly on $[-1, 1]$ with only n evaluations of f .

To illustrate, the 2-point Gaussian rule on $[-1, 1]$ picks

$$x_1 = -\frac{1}{\sqrt{3}}, \quad x_2 = \frac{1}{\sqrt{3}}, \quad w_1 = w_2 = 1,$$

so that

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right),$$

which integrates any cubic polynomial exactly.

Table of Nodes and Weights. The following table lists the Gauss–Legendre nodes x_i and weights w_i on the standard interval $[-1, 1]$ for $n = 2, 3, 4, 5$, as tabulated in standard references [4]. These are the constants you plug into the formula above; the $n = 5$ row is the one my implementation actually uses.

n	x_i (nodes)	w_i (weights)
2	$-\frac{1}{\sqrt{3}} \approx -0.577350$	1
	$+\frac{1}{\sqrt{3}} \approx 0.577350$	1
3	$-\sqrt{\frac{3}{5}} \approx -0.774597$	$\frac{5}{9} \approx 0.555556$
	0	$\frac{8}{9} \approx 0.888889$
	$+\sqrt{\frac{3}{5}} \approx 0.774597$	$\frac{5}{9} \approx 0.555556$

n	x_i (nodes)	w_i (weights)
4	−0.861136	0.347855
	−0.339981	0.652145
	+0.339981	0.652145
	+0.861136	0.347855
5	−0.906180	0.236927
	−0.538469	0.478629
	0	128/225 $\approx$ 0.568889
	+0.538469	0.478629
	+0.906180	0.236927

Application to Arc Length. In our arc-length integral, set

$$f(\tau) = \|\mathbf{r}'(\tau)\|, \quad a = 0, \quad b = t,$$

and use the above x_i, w_i with

$$\tau_i = \frac{b-a}{2} x_i + \frac{a+b}{2} = \frac{t}{2} x_i + \frac{t}{2},$$

to compute

$$s(t) \approx \frac{t}{2} \sum_{i=1}^n w_i \|\mathbf{r}'(\tau_i)\|.$$

Why one panel is not enough. For a cubic Bézier the integrand $\|\mathbf{r}'(\tau)\|$ is the *square root* of a polynomial, not a polynomial itself, so the “exact for degree $2n - 1$ ” guarantee does not apply at all — no choice of n makes a single application of the rule exact here. What the rule still gives is fast convergence for smooth integrands, and that convergence is much faster on short intervals than on long ones. On a long or wiggly curve a single $n = 5$ panel spanning all of $[0, t]$ leaves error large enough to misplace the robot; the fix is *composite* quadrature, splitting the interval into m equal panels and applying the rule on each [3]:

$$s(t) \approx \sum_{j=0}^{m-1} \frac{h}{2} \sum_{i=1}^n w_i \left\| \mathbf{r}' \left(hj + \frac{h}{2}(x_i + 1) \right) \right\|, \quad h = \frac{t}{m}.$$

I use $n = 5$ nodes with $m = 8$ panels, so each call costs 40 evaluations of $\|\mathbf{r}'\|$ — cheap enough to run every control cycle, and accurate enough that the residual error is well below the robot’s odometry noise. Accuracy still degrades near degenerate curves (control-point placements where $\|\mathbf{r}'\|$ approaches zero, creating a cusp), where further subdivision is the remedy [3].

6 Finding the Next Parameter: Newton–Raphson

To move a small distance Δs along the path, we need to find the new parameter t_{next} such that

$$s(t_{\text{next}}) = s_{\text{current}} + \Delta s.$$

We use Newton–Raphson (a root-finding method) to solve this equation, the same combination of Gaussian quadrature and Newton iteration used in [3]:

$$t_{k+1} = t_k - \frac{s(t_k) - (s_{\text{current}} + \Delta s)}{\|\mathbf{r}'(t_k)\|}.$$

Why Newton's method? We want to solve

$$F(t) = s(t) - (s_{\text{current}} + \Delta s) = 0$$

for t . Newton–Raphson is an efficient iterative technique that uses the derivative $F'(t) = s'(t) = \|\mathbf{r}'(t)\|$. Note that this is the *parametric* speed of the curve (how fast the point moves as t changes), not the robot's commanded velocity—we reserve v for the latter to avoid confusion. Starting from an initial guess t_k , we linearize and obtain

$$t_{k+1} = t_k - \frac{F(t_k)}{F'(t_k)} = t_k - \frac{s(t_k) - (s_{\text{current}} + \Delta s)}{\|\mathbf{r}'(t_k)\|}.$$

In practice each iterate should also be clamped to the valid parameter range $t \in [0, 1]$; if Newton's method fails to converge within k_{max} iterations (possible when $\|\mathbf{r}'\|$ is very small), falling back to bisection is a robust remedy [3].

Stopping criterion. We iterate $k = 0, 1, 2, \dots$ until one of the following is met:

$$|t_{k+1} - t_k| < \varepsilon \quad \text{or} \quad |s(t_k) - (s_{\text{current}} + \Delta s)| < \varepsilon \quad \text{or} \quad k \geq k_{\text{max}},$$

where $\varepsilon > 0$ is a chosen tolerance (e.g. 10^{-6}) and k_{max} (e.g. 10) caps the number of iterations to avoid infinite loops. This ensures we stop once the update or the residual is sufficiently small, or when we've reached a safe iteration limit.

7 Acceleration and Deceleration Distances

From basic kinematics:

$$v^2 = v_0^2 + 2a \Delta s \implies \Delta s = \frac{v^2 - v_0^2}{2a},$$

which tells us the distance needed to change speed under constant acceleration a .

- If $a > 0$: Δs is the distance needed to **accelerate** from v_0 up to v .
- If $a < 0$: Δs is the distance needed to **decelerate** (brake) from v_0 down to v .

Why it works. This formula is derived by integrating acceleration over distance:

$$a = \frac{dv}{dt} \implies v dv = a ds \implies \int_{v_0}^v v dv = \int_0^{\Delta s} a ds \implies \frac{v^2 - v_0^2}{2} = a \Delta s,$$

so rearranging gives $\Delta s = \frac{v^2 - v_0^2}{2a}$.

8 Curvature and Turning Speed

For a 2D path $\mathbf{r}(t) = (x(t), y(t))$, the *signed* curvature is defined as:

$$\kappa(t) = \frac{x'(t)y''(t) - y'(t)x''(t)}{(x'(t)^2 + y'(t)^2)^{3/2}}, \quad R(t) = \frac{1}{|\kappa(t)|},$$

where $R(t)$ is the radius of curvature at that point on the path. The sign of κ tells us the turn direction (positive for a left turn, negative for a right turn, with the usual counterclockwise convention). The speed limit below only depends on $|\kappa|$, but the sign matters later: the commanded angular velocity is $\omega = v\kappa$, so if we dropped the sign the robot could only ever turn one way.

What to do at a cusp. The denominator $\|\mathbf{r}'(t)\|^3$ collapses to zero where the parametric speed vanishes, and the formula is undefined there. It is tempting to return $\kappa = 0$ to avoid the division, but that is exactly backwards: a cusp is the *sharpest* point on the path, geometrically a turn of zero radius, so $|\kappa| \rightarrow \infty$. Returning zero would remove the curvature speed limit precisely where it is needed most and send the robot through the cusp at full speed. The safe convention is to saturate $|\kappa|$ at a large finite value (keeping the sign of the cross product where it is still recoverable, so the turn direction survives). The limit $v_{\text{curve}} = v_{\text{max}} \cdot R/(R + w/2)$ then drives $v \rightarrow 0$ as $R \rightarrow 0$, and since $\omega = v\kappa$ the angular velocity tends to $\omega \rightarrow 2v_{\text{max}}/w$ – the robot turning in place at the drivetrain’s maximum rate, which is the physically correct behaviour at a cusp. The same degeneracy breaks the Newton iteration of the previous section, because $\|\mathbf{r}'\|$ is its derivative; that is when the bisection fallback earns its keep.

The wheels of a differential drive robot have a maximum speed. In a turn, the outer wheel travels faster than the robot’s center, so the center must slow down to keep the outer wheel within its limit [5], [6]. We therefore restrict speed using:

$$v_{\text{curve}(t)} = v_{\text{max}} \cdot \frac{R(t)}{R(t) + \frac{w}{2}},$$

where w is the track width (distance between the left and right wheels).

Why it works. We start with the differential drive kinematic equations [5]:

$$V_l = v - r \cdot \omega \quad \text{and} \quad V_r = v + r \cdot \omega,$$

where v is the linear velocity, ω is the angular velocity, and r is half the track width (the distance from the center of the robot to either wheel).

We also know that $\omega = v \cdot \kappa$, by the definition of angular velocity, where κ is the curvature of the path. Substituting into the equations and factoring gives:

$$V_l = v \cdot (1 - r \cdot \kappa), \quad V_r = v \cdot (1 + r \cdot \kappa).$$

Each wheel must stay within its maximum velocity, so we create two inequalities:

$$|v \cdot (1 - r \cdot \kappa)| \leq v_{\text{max}}, \quad |v \cdot (1 + r \cdot \kappa)| \leq v_{\text{max}}.$$

Since $v \geq 0$, we can take v out of the absolute value and divide both sides:

$$\max(|1 - r \cdot \kappa|, |1 + r \cdot \kappa|) \leq \frac{v_{\text{max}}}{v}.$$

Noting that the maximum of $|1 - r \cdot \kappa|$ and $|1 + r \cdot \kappa|$ is always $1 + |r \cdot \kappa|$, we simplify to:

$$1 + |r \cdot \kappa| \leq \frac{v_{\text{max}}}{v} \quad \Rightarrow \quad v \leq \frac{v_{\text{max}}}{1 + |r \cdot \kappa|}.$$

This tells us how fast the center of the robot can move without exceeding the speed limit of either wheel during a turn.

Now, since $|\kappa| = \frac{1}{R(t)}$, where $R(t)$ is the radius of curvature of the path, we can substitute $|r \cdot \kappa| = \frac{r}{R(t)}$. Plugging this into the inequality gives:

$$v \leq \frac{v_{\max}}{1 + \frac{r}{R(t)}} = v_{\max} \cdot \frac{R(t)}{R(t) + r}.$$

This gives the final form:

$$v_{\text{curve}(t)} = v_{\max} \cdot \frac{R(t)}{R(t) + r},$$

which is a more geometric way of expressing the speed limit based on curvature and robot geometry.

9 Keyframes: Localisation and Velocity Interpolation

9.1 Locating a Keyframe on the Path

A path editor hands me keyframes as (x, y, v) : a point the user clicked on the field and the speed they want there. Everything downstream is indexed by the curve parameter t , so the first job is to turn that point $\mathbf{p} = (x, y)$ into a parameter.

The tempting shortcut is to solve $x(t) = x_{\text{kf}}$ for t and ignore y . That is wrong on any path that revisits an x coordinate — an S-curve, a loop, or anything that doubles back — because the equation then has several roots and nothing in it distinguishes the intended one from a branch of the path metres away. It also fails outright when the keyframe sits slightly off the curve, since $x(t) = x_{\text{kf}}$ may have no solution in $[0, 1]$ at all.

The correct question is a projection: which point of the curve is closest to $\mathbf{p}$?

$$t^* = \underset{t \in [0,1]}{\operatorname{argmin}} \|\mathbf{r}(t) - \mathbf{p}\|^2.$$

Differentiating the objective gives the stationary condition

$$\frac{d}{dt} \|\mathbf{r}(t) - \mathbf{p}\|^2 = 2(\mathbf{r}(t) - \mathbf{p}) \cdot \mathbf{r}'(t) = 0,$$

which says the error vector is orthogonal to the tangent — the familiar foot-of-perpendicular condition. Writing $g(t) = (\mathbf{r}(t) - \mathbf{p}) \cdot \mathbf{r}'(t)$, Newton's method applies again with

$$g'(t) = \|\mathbf{r}'(t)\|^2 + (\mathbf{r}(t) - \mathbf{p}) \cdot \mathbf{r}''(t), \quad t_{k+1} = t_k - \frac{g(t_k)}{g'(t_k)}.$$

For a cubic Bézier, g is a quintic in t and can have up to five roots, so the squared distance is *not* convex and Newton alone will happily converge to the wrong local minimum. I therefore scan a coarse grid (64 samples) for the best basin, refine from there, and discard the refinement if it ends up farther from $\mathbf{p}$ than the sampled seed.

Two things are worth rejecting rather than silently accepting:

- **Off-path keyframes.** The projection always returns some t^* , even for a point nowhere near the curve. If the residual $\|\mathbf{r}(t^*) - \mathbf{p}\|$ exceeds a tolerance, the keyframe's velocity would be applied at a location the user never intended, so it is better to raise an error than to guess.

- **Out-of-order keyframes.** The interpolation below walks a list sorted by position along the path, so a keyframe projecting to a t^* behind its predecessor would bracket the wrong interval. Requiring t^* to be non-decreasing catches this at setup time.

Given t^* for each keyframe, the arc-length position follows from the quadrature of the previous section: $s_i = s(t_i^*)$.

9.2 Interpolating Between Keyframes

Now we have pairs (s_i, v_i) : desired speeds v_i at arc lengths s_i . For an intermediate position s with $s_i \leq s \leq s_{i+1}$, define the fraction of the way through the interval

$$\lambda = \frac{s - s_i}{s_{i+1} - s_i} \in [0, 1].$$

The obvious move is to interpolate v itself linearly in s . It is worth seeing why that is the wrong quantity to make linear. Along the path $a = v \frac{dv}{ds}$, so a profile linear in s implies

$$a(s) = v(s) \cdot \frac{v_{i+1} - v_i}{s_{i+1} - s_i},$$

which is proportional to the current speed: the segment demands the *most* acceleration exactly where the robot is already moving fastest, and the demand changes continuously across an interval the planner treats as one piece.

Interpolating in v^2 instead fixes this:

$$v_{\text{interp}(s)}^2 = v_i^2 + \lambda (v_{i+1}^2 - v_i^2), \quad v_{\text{interp}(s)} = \sqrt{v_i^2 + \lambda (v_{i+1}^2 - v_i^2)}.$$

Compare this with the kinematic relation $v^2 = v_0^2 + 2a(s - s_0)$ from Section 7. They match term for term with

$$a = \frac{v_{i+1}^2 - v_i^2}{2(s_{i+1} - s_i)},$$

a *constant*. So interpolating in v^2 makes each keyframe interval a constant-acceleration segment, which is the same shape as every other limit in the planner and therefore something the acceleration limit can actually honour. In code, clamp λ to $[0, 1]$ so a robot slightly past a keyframe does not extrapolate, and floor the radicand at zero before taking the square root, since rounding can push it slightly negative when both endpoint speeds are near zero.

10 Velocity Planning Algorithm

Ok, now we put it all together. We now follow Section 2, which describes our goals. At every time step we compute several candidate speed limits and command the smallest one:

1. Compute v_{max} : the robot's absolute maximum speed (a constant limit).
2. Compute v_{accel} : the speed limit imposed by the maximum acceleration. Since the robot's speed can only change by $a_{\text{max}} \Delta t$ in one time step, we have $v_{\text{accel}} = v(t) + a_{\text{max}} \Delta t$.

3. Compute v_{brake} : the speed limit that guarantees the robot can still decelerate to the exit velocity v_{end} by the end of the path. First, the braking distance needed to slow from v_{max} to v_{end} at maximum deceleration follows from the kinematic equation in Section 7 (with $a_{\text{max}} > 0$):

$$d_{\text{decel}} = \frac{v_{\text{max}}^2 - v_{\text{end}}^2}{2 \cdot a_{\text{max}}}.$$

While the remaining distance $d_{\text{remaining}}$ exceeds d_{decel} , braking imposes no limit. Once $d_{\text{remaining}} \leq d_{\text{decel}}$, we need the largest speed v from which the robot can still reach v_{end} within $d_{\text{remaining}}$ (assuming a trapezoidal profile). Applying the same kinematic equation over the remaining distance:

$$v_{\text{end}}^2 = v^2 - 2 \cdot a_{\text{max}} \cdot d_{\text{remaining}} \implies v_{\text{brake}} = \sqrt{v_{\text{end}}^2 + 2 \cdot a_{\text{max}} \cdot d_{\text{remaining}}}.$$

This is exactly the deceleration ramp of a trapezoidal profile, expressed as a function of distance-to-go: at $d_{\text{remaining}} = d_{\text{decel}}$ it equals v_{max} , and at $d_{\text{remaining}} = 0$ it equals v_{end} .

4. Compute v_{curve} : the speed limit due to curvature at the robot's current position (using the turning formula).
5. Compute v_{interp} : the speed dictated by keyframe interpolation at the current position.

All of these are upper bounds, so the commanded speed is simply their minimum:

$$v_{\text{desired}} = \min\{v_{\text{max}}, v_{\text{accel}}, v_{\text{brake}}, v_{\text{curve}}, v_{\text{interp}}\}.$$

A limitation to be aware of. This planner is *greedy*: it only looks at the constraints at the robot's current position (plus the braking constraint toward the path's end). If a sharp turn or a slow keyframe lies ahead mid-path, nothing forces the robot to start braking for it early—by the time v_{curve} or v_{interp} drops, the robot may be physically unable to slow down fast enough, since real deceleration is also bounded by a_{max} . The standard fix is to precompute the profile over a discretized path with a *forward pass* (propagating acceleration limits) and a *backward pass* (propagating deceleration limits back from every slow point), taking the pointwise minimum. This two-pass idea goes back to the time-optimal path parameterization literature [7], [8], [9] and is exactly what tools like WPILib's trajectory generator do [6]; see Section 16.

11 Handling Multiple Path Segments

Many paths consist of several curve segments (splines) joined end-to-end. We can apply the motion profiling approach to each segment in sequence, while ensuring smooth transitions between segments.

At the end of one segment, the next segment begins with the robot continuing at whatever speed it reached. In practice, the end of one spline and the start of the next share a common point and velocity (if planned correctly). To handle the transition smoothly:

- If the remaining distance in the current segment is greater than the distance Δs the robot will travel in the next time step, then the robot simply continues within the same segment (it hasn't reached the end yet).
- If Δs would carry the robot past the end of the current segment, we first move it to the end of that segment (using the portion of Δs needed to cover the last bit of that spline). Then, we use the leftover distance to continue into the next segment (starting the next segment's arc length from 0 at its beginning).

This way, the robot seamlessly continues from one spline to the next. The parameter t resets to 0 at the start of the new segment, and we proceed with the same calculations on the new segment. (If using keyframes, you can also set a keyframe at the segment boundary to enforce a specific speed at that point.)

12 Simplified C++ Example from VMPLib

```
void TrapezoidalProfile::step() {
    ++step_count_;
    time_accum_ += dt_;
    s_current_ = sFunction(control_, prev_t_);

    float keyframe_lim = computeKeyframeLimit();
    float curvature_lim = computeCurvatureVelocityLimit(prev_t_);
    float accel_lim = computeAccelerationLimit();
    float brake_lim = computeBrakingLimit(s_current_);

    // All limits are upper bounds; command the smallest.
    float desired_linear = std::min({ curvature_lim,
                                      accel_lim,
                                      brake_lim,
                                      keyframe_lim,
                                      max_lin_vel_ });

    float deltaS = desired_linear * dt_;
    float next_t = findNextT(s_current_, deltaS);

    float kappa = signedCurvature(control_, next_t);
    float turning_component = kappa * desired_linear;

    poses_.push_back(findXandY(control_, next_t));
    velocities_.push_back({ desired_linear, turning_component, time_accum_ });

    prev_t_ = next_t;
    cur_speed_ = desired_linear;
}
```

Each call advances the profile by one timestep and appends to the pose and velocity vectors rather than returning a sample; the caller loops on `isFinished()` and reads the accumulated trajectory afterwards. That termination test checks a step counter as well as $t \geq 1$, so a path that stalls near a cusp ends instead of spinning forever.

How to use these outputs. Send v_{desired} and ω to your drivetrain's velocity controller (commonly a PID or feedforward velocity controller, which is beyond the scope of this guide). A typical differential-drive conversion is:

$$v_{\text{left}} = v_{\text{desired}} - \frac{\omega w}{2}, \quad v_{\text{right}} = v_{\text{desired}} + \frac{\omega w}{2},$$

where w is the track width. That is open loop: it assumes the robot ends up where the plan says it does. To close the loop on pose feedback, wrap it in a tracking controller such as pure pursuit [10] or RAMSETE, which the next section derives.

13 Trajectory Tracking with RAMSETE

The profile above produces a reference trajectory: a pose $\mathbf{q}_d = (x_d, y_d, \theta_d)$ and a velocity pair (v_d, ω_d) at each timestep. Wheel slip, odometry drift, and modelling error all mean the robot will not sit exactly on that reference, so a tracking controller corrects the difference. RAMSETE is the standard choice for a

differential drive [11]: it is a nonlinear controller with only two tuning parameters and a global convergence proof.

Error in the robot's frame. The pose error is expressed in the robot's own frame rather than the field frame, which is what lets one control law work at any heading:

$$\begin{pmatrix} e_x \\ e_y \\ e_\theta \end{pmatrix} = \begin{pmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_d - x \\ y_d - y \\ \theta_d - \theta \end{pmatrix}.$$

So e_x is the along-track error (ahead of the robot is positive), e_y the cross-track error (to the robot's left), and e_θ the heading error, which must be wrapped to $(-\pi, \pi]$ before use.

The control law. RAMSETE commands [11], [12]

$$\begin{aligned} k &= 2\zeta\sqrt{\omega_d^2 + b v_d^2}, \\ v &= v_d \cos e_\theta + k e_x, \\ \omega &= \omega_d + k e_\theta + b v_d \text{sinc}(e_\theta) e_y, \end{aligned}$$

where $\text{sinc}(e_\theta) = \sin(e_\theta)/e_\theta$, taken as 1 in the limit $e_\theta \rightarrow 0$ (in code, switch to 1 below a small threshold to avoid 0/0). The gains are $b > 0$ and $\zeta \in (0, 1)$; larger b makes the correction more aggressive and ζ sets the damping. Note the structure: the linear command corrects along-track error directly, while cross-track error is corrected through *steering* (e_y appears only in ω), which is the only way a differential drive can fix it — it cannot translate sideways.

Mind the units. This is the part I got wrong the first time, and dimensional analysis is what caught it. Since k multiplies e_θ (radians, i.e. dimensionless) to produce an angular velocity, k must carry units of s^{-1} . Inside the square root, ω_d^2 is already s^{-2} , so $b v_d^2$ must also be s^{-2} ; with v_d in m / s that forces

$$[b] = \text{m}^{-2}.$$

Everything else follows: $k e_x$ has units $\text{s}^{-1} \cdot \text{m} = \text{m}/\text{s}$, matching v , and $b v_d e_y$ has units $\text{m}^{-2} \cdot \text{m}/\text{s} \cdot \text{m} = \text{s}^{-1}$, matching ω . Any arrangement of the gains that fails this check is wrong regardless of how plausible it looks or how well it happens to behave in one test — which is exactly how I found the bug in my original implementation.

A Lyapunov argument establishes that the origin $e = \mathbf{0}$ is asymptotically stable for any $b > 0$, $\zeta \in (0, 1)$; see [11] for the original proof and [12] for a derivation aimed at competition robotics. One practical caveat: because k scales with v_d and ω_d , the controller has no authority when the reference is stationary, so RAMSETE corrects error while the robot is moving and should be handed off to a pose controller at rest.

14 Why Acceleration/Deceleration Distance is an Approximation

The constant- a distance formulas we derived assume the robot accelerates and then decelerates in a perfect “trapezoidal” velocity profile (speeding up, cruising at $v_{\max}$, then slowing down). This assumes the robot has a long enough straight run to reach and maintain that top speed.

However, along a curvy path, we impose extra speed limits based on curvature. This often prevents the robot from ever reaching the planned $v_{\max}$ on that segment of the path. In other words, the velocity profile gets “cut off” or dips in the middle due to a turn. The robot might have to start slowing down for the

turn before it ever fully accelerates to $v_{\max}$. The figure below illustrates this: in the curved case, the robot cannot maintain a flat-top speed profile because it must slow down for the turn.

Because of this effect, the simple distance formulas for accelerating and braking become approximations. In practice, the robot might start decelerating earlier (or not accelerate as much) when a turn is coming up, meaning our calculated distances might slightly overestimate how far it needs to speed up or slow down.

To improve accuracy, we can account for curvature in our motion profile calculations. For example:

- Instead of relying solely on the constant- a formula, we could integrate the actual velocity curve (which already factors in curvature limits) to find the distance covered during acceleration and deceleration.
- We can dynamically adjust target speeds. If a sharp turn is imminent, the algorithm can cap the speed below $v_{\max}$ ahead of time, so that the robot never accelerates beyond what it can comfortably slow down from when it reaches the curve. Done systematically over a discretized path, this is the forward/backward-pass approach mentioned at the end of the velocity planning section [7], [9].

15 Ideas for Future Improvement

- **Motor Model:** Acceleration decreases with speed (motors produce less torque at high RPM). For example:

$$a(v) = a_{\max} \left(1 - \frac{v}{v_{\text{free}}} \right),$$

where v_{free} is the motor's free (no-load) speed.

- **Full Dynamics:** Model the robot's full dynamics (mass, rotational inertia) for more realistic acceleration behavior possibly including friction and maybe even drag.
- **Implement back propagation:** Propagate deceleration limits back from every slow point, taking the pointwise minimum at each.

16 Related Work and Similar Systems

The approach in this guide was developed independently for VMPLib, but each ingredient—and the overall pipeline—has well-established relatives worth knowing about.

Arc-length parameterization. Wang, Kearney, and Atkinson [3] describe essentially the same numerical machinery used here (Gauss–Legendre quadrature for $s(t)$, Newton–Raphson to invert it) for driving simulators, where vehicles must move along spline roads at prescribed speeds. Their paper is a good reference for accuracy and robustness details we glossed over.

Time-optimal path parameterization (TOPP). The academic formulation of “given a path and actuator limits, find the fastest feasible speed profile” dates to Bobrow et al. [7] and Shin and McKay [8], who proved the optimal profile alternates between maximum acceleration and maximum deceleration segments. Modern solvers such as TOPP-RA [9] compute the profile with a backward *controllable-set* pass followed by a forward pass—the rigorous version of the forward/backward-pass fix discussed in the velocity planning section.

Spline trajectories for differential drives. Sprunk's thesis [13] is the closest academic sibling of this guide: it plans Bézier-spline paths for a differential drive robot and computes velocity profiles respecting translational, rotational, and centripetal limits, with curvature-continuous joins between segments. See also the companion paper by Lau, Sprunk, and Burgard [14].

Competition robotics implementations. The FIRST Robotics Competition community has converged on the same architecture. Team 254’s influential presentation *Motion Planning and Control in FRC* [15] popularized spline paths with trapezoidal profiles; WPILib’s trajectory generator [6] performs a forward/backward pass subject to constraints, including a differential-drive wheel-speed constraint identical in spirit to our curvature limit; and GUI tools such as PathPlanner [16] and PATH.JERRYIO [2] provide the keyframe-editing workflow described earlier. Veness’s free book [12] gives an accessible derivation of the RAMSETE controller stated above.

17 Further Learning Resources

- **YouTube** - “**The Continuity of Splines**”: Deep dive into spline curves by Freya Holmér (Highly Reccomend!!!!) [17]
- **Khan Academy**: Derivatives and Integrals (introductory calculus lessons)
- **Feynman Lectures on Physics**, Vol. 1 (insightful treatment of motion in Chapter 8)
- **MIT OCW**: 18.01 Single Variable Calculus (free online course materials)
- **MIT OCW**: 18.02 Multi Variable Calculus for an in-depth look at some of the Calculus going on including possibly derivations of curvature and such (free online course materials as well)

Bibliography

- [1] G. Farin, *Curves and Surfaces for CAGD: A Practical Guide*, 5th ed. Morgan Kaufmann, 2002.
- [2] J. Lum, “PATH.JERRYIO: A Path Editor for VEX and FTC Robots.” [Online]. Available: <https://path.jerryio.com/>
- [3] H. Wang, J. Kearney, and K. Atkinson, “Arc-Length Parameterized Spline Curves for Real-Time Simulation,” in *Proceedings of the 5th International Conference on Curves and Surfaces*, Saint-Malo, France, 2002, pp. 387–396. [Online]. Available: <https://homepage.math.uiowa.edu/~atkinson/ftp/CurvesAndSurfacesArcLength.pdf>
- [4] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. in Applied Mathematics Series 55. National Bureau of Standards, 1964.
- [5] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006. [Online]. Available: <http://lavalle.pl/planning/>
- [6] WPILib, “Trajectory Generation and Constraints.” [Online]. Available: <https://docs.wpilib.org/en/stable/docs/software/advanced-controls/trajectories/trajectory-generation.html>
- [7] J. E. Bobrow, S. Dubowsky, and J. S. Gibson, “Time-Optimal Control of Robotic Manipulators Along Specified Paths,” *The International Journal of Robotics Research*, vol. 4, no. 3, pp. 3–17, 1985.
- [8] K. G. Shin and N. D. McKay, “Minimum-Time Control of Robotic Manipulators with Geometric Path Constraints,” *IEEE Transactions on Automatic Control*, vol. 30, no. 6, pp. 531–541, 1985.
- [9] H. Pham and Q.-C. Pham, “A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis,” *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 645–659, 2018, [Online]. Available: <https://github.com/hungpham2511/toppra>
- [10] R. C. Coulter, “Implementation of the Pure Pursuit Path Tracking Algorithm,” technical report CMU-RI-TR-92-01, 1992. [Online]. Available: https://www.ri.cmu.edu/pub_files/pub3/coulter_r_craig_1992_1/coulter_r_craig_1992_1.pdf
- [11] A. De Luca, G. Oriolo, and M. Vendittelli, “Control of Wheeled Mobile Robots: An Experimental Overview,” in *RAMSETE: Articulated and Mobile Robotics for Services and Technologies*, vol. 270, S. Nicosia, B. Siciliano, A. Bicchi, and P. Valigi, Eds., in Lecture Notes in Control and Information Sciences, vol. 270., Springer, 2001.
- [12] T. Veness, *Controls Engineering in the FIRST Robotics Competition*. Self-published, 2024. [Online]. Available: <https://controls-in-frc.link/>
- [13] C. Sprunk, “Planning Motion Trajectories for Mobile Robots Using Splines,” Student project report, 2008. [Online]. Available: <http://www2.informatik.uni-freiburg.de/~lau/students/Sprunk2008.pdf>
- [14] B. Lau, C. Sprunk, and W. Burgard, “Kinodynamic Motion Planning for Mobile Robots Using Splines,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2009, pp. 2427–2433.
- [15] J. Russell, T. Bottiglieri, and A. Schuh, “Motion Planning and Control in FRC.” [Online]. Available: <https://www.youtube.com/watch?v=8319J1BEHwM>

- [16] M. Jansen, “PathPlanner: A Motion Profile Generator for FRC Robots.” [Online]. Available: <https://github.com/mjansen4857/pathplanner>
- [17] F. Holmér, “The Continuity of Splines.” [Online]. Available: <https://www.youtube.com/watch?v=jvPPXbo87ds>